МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ» (ДГТУ)

Факультет
Информатика и вычислительная техника

Кафедра
Программное обеспечение вычислительной техники и автоматизированных систем

Способы повышения качества обучения

Практикум

по выполнению лабораторной работы №2
по дисциплине «Нейронные сети и глубокое обучение»

Большие данные и машинное обучение
(указывается направленность (профиль) образовательной программы)

02.03.03 Математическое обеспечение и администрирование информационных систем
(указывается код и наименование направления подготовки)

Ростов-на-Дону
2026 г.

Составители: канд. техн. наук, доц. каф. А.А. Скляренко

УДК 004.432

Многослойная нейронная сеть: метод. указания. – Ростов н/Д: Издательский центр ДГТУ, 2026.

В методическом указании рассматриваются способы повышения качества обучения модели, создания устойчивых моделей. Приведены задания к лабораторной работе, помогающие закрепить на практике полученные знания, и контрольные вопросы для самопроверки.
Предназначено для обучающихся по направлению 02.03.02 «Математическое обеспечение и администрирование информационных систем», профиль «Большие данные и машинное обучение».

Ответственный за выпуск:
зав. кафедрой «Программное обеспечение вычислительной техники и автоматизированных систем» Долгов В.В.

© А.А. Скляренко,
2026
© Издательский центр ДГТУ, 2026

1 Теоретическая часть

1.1 Переобучение

Переобучение — ситуация, когда модель хорошо запоминает обучающие данные, но плохо обобщает на новые (тестовые).

Признаки переобучения:

- высокая точность на обучающей выборке и низкая — на тестовой;
- большой разрыв между потерями (loss) на обучающей и валидационной выборках;
- сложность модели значительно превышает сложность данных.

Причины:

- слишком сложная архитектура сети;
- недостаточный объём данных;
- отсутствие регуляризации;
- длительное обучение.

Методы борьбы с переобучением

L1-регуляризация (Lasso):

- добавляет к функции потерь штраф, пропорциональный сумме абсолютных значений весов;
- приводит к разреженным моделям (многие веса становятся нулевыми);
- полезна для отбора признаков.

L2-регуляризация (Ridge):

- штрафует за большие значения весов;
- делает веса более «гладкими», предотвращает резкие скачки;
- чаще применяется в нейронных сетях.

Пример добавления регуляризации:

```
from tensorflow.keras.regularizers import l2

l2_model = Sequential([
    Dense(128, activation='relu',
          kernel_regularizer=l2(0.01),  # L2-штраф λ=0.01
          input_shape=(784,)),
    Dense(64, activation='relu', kernel_regularizer=l2(0.01)),
    Dense(10, activation='softmax')
```

```
])
```

Dropout:

- во время обучения случайно «отключает» часть нейронов (с заданной вероятностью);
- заставляет сеть не полагаться на конкретные связи;
- на тесте все нейроны активны.

Пример добавления Dropout

```
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization

dropout_model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dropout(0.3),  # Отключаем 30% нейронов
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(10, activation='softmax')
])
```

Batch Normalization:

- нормализует выходы слоёв: приводит к нулевому среднему и единичной дисперсии;
- стабилизирует обучение, позволяет использовать большие скорости обучения;
- имеет лёгкий регуляризационный эффект.

Пример добавления нормализации

```
from tensorflow.keras.layers import BatchNormalization

batchnorm_model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    BatchNormalization(),  # Нормализация выходов слоя
    Dense(64, activation='relu'),
    BatchNormalization(),
    Dense(10, activation='softmax')
     ])
```

Ранняя остановка (Early Stopping):

- останавливает обучение, когда ошибка на валидационной выборке перестаёт уменьшаться;
- предотвращает переобучение за счёт ограничения числа эпох;
- реализуется через callback в Keras.

Пример ранней остановки

```
from tensorflow.keras.callbacks import EarlyStopping

early_stopping = EarlyStopping(
    monitor='val_loss',      # Мониторим валидационную ошибку
    patience=5,           # Ждём 5 эпох без улучшения
    restore_best_weights=True,  # Восстанавливаем лучшие веса
    verbose=1
     )
history_earlystop = model_with_earlystop.fit(
    x_train, y_train,
    epochs=50,  # Задаём большое число эпох
    batch_size=128,
    validation_split=0.2,
    callbacks=[early_stopping],
    verbose=1
)
```

1.2 Кросс-валидация (Cross-Validation)

Кросс-валидация — метод оценки обобщающей способности модели путём многократного разбиения данных на обучающую и валидационную выборки.

k-fold кросс-валидация:

- данные делятся на k равных частей (фолдов);
- модель обучается раз, каждый раз используя один фолд как валидационный, остальные — как обучающие;
- итоговая оценка — среднее значение метрики по всем фолдам.

Преимущества перед одним разделением train/test:

- более надёжная оценка качества (используются все данные);
- меньше зависимость от случайного разбиения;
- позволяет оценить вариативность результатов (через стандартное отклонение).

Пример:

```
from sklearn.model_selection import KFold

kfold = KFold(n_splits=5, shuffle=True, random_state=42)

for train_idx, val_idx in kfold.split(x_cv):
    # Разделение данных
    x_fold_train, x_fold_val = x_cv[train_idx], x_cv[val_idx]
    y_fold_train, y_fold_val = y_cv[train_idx], y_cv[val_idx]
```

```
    # Создание и обучение модели
    fold_model = create_model()
    fold_model.fit(x_fold_train, y_fold_train,
                   epochs=10, batch_size=32, verbose=0)

    # Оценка на валидационном фолде
        scores = fold_model.evaluate(x_fold_val, y_fold_val,
     verbose=0)
```

1.3 Подбор гиперпараметров

Гиперпараметры — параметры, задающиеся до обучения (в отличие от весов, которые настраиваются в процессе):

- скорость обучения (learning rate);
- размер батча (batch size);
- число слоёв/нейронов;
- коэффициенты регуляризации;
- вероятность Dropout и т. д.

Методы поиска:

Random Search (случайный поиск):

- выбирает случайные комбинации гиперпараметров;
- часто находит хорошие решения быстрее, чем Grid Search;
- эффективен, когда лишь некоторые гиперпараметры сильно влияют на результат.

Пример

```
from scipy.stats import uniform
from sklearn.model_selection import RandomizedSearchCV
# Пространство гиперпараметров
param_distributions = {
    'learning_rate': uniform(0.0001, 0.01),
    'dropout_rate': [0.2, 0.3, 0.5]
}
# Обертка для использования с sklearn
from tensorflow.keras.wrappers.scikit_learn import KerasClassifier
model_wrapper = KerasClassifier(build_fn=build_model, epochs=10,
batch_size=32, verbose=0)
random_search = RandomizedSearchCV(
    estimator=model_wrapper,
    param_distributions=param_distributions,
    n_iter=10,  # Число случайных комбинаций
```

```
    cv=1,      # 1-fold кросс
    scoring='accuracy',
    verbose=1
)

# Запуск поиска
random_search.fit(x_cv, y_cv)

# Результаты
print("Лучшие гиперпараметры:", random_search.best_params_)
print("Лучшая точность:", random_search.best_score_)
```

Grid Search (полный перебор):

- перебирает все возможные комбинации из заданного набора значений;
- гарантирует нахождение лучшего решения в заданном пространстве;
- вычислительно затратен при большом числе гиперпараметров.

Пример

```
from sklearn.model_selection import GridSearchCV

grid_search = GridSearchCV(
    estimator=model_wrapper,
    param_grid=param_grid,
    cv=3,
    scoring='accuracy',
    verbose=1
)

# Запуск Grid Search
grid_search.fit(x_cv, y_cv)

print("Лучшие гиперпараметры (Grid Search):",
grid_search.best_params_)
print("Лучшая точность (Grid Search):", grid_search.best_score_)
```

2 Задание к лабораторной работе

В последующих задания взять датасет и нейронную сеть из второго задания первой лабораторной работы. Выполнить ее обучение, в результате получится базовая модель (если вы в нее уже добавляли слои Dropout, убрать их перед запуском обучения, а также можно увеличить количество нейронов в слоях, чтобы увеличить

шанс переобучения модели). Все сравнения приводить с помощью визуализации графиков обучения по точности и ошибки.

Задание 1

Оценить влияние регуляризации на качество обучения модели и борьбы с переобучением.

1. Добавьте L2-регуляризацию к весам слоёв.
2. Обучите модель с разными значениями коэффициента регуляризации (0.01, 0.1).
3. Сравните результаты с базовой моделью.

Задание 2

Оценить влияния слоя Dropout на качество обучения модели и борьбы с переобучением.

Слои Dropout добавляем к базовой модели, убрав регуляризацию из 1 задания.

1. Вставьте слои Dropout после полносвязных слоёв (вероятность отсева 0.2, 0.4).
2. Оцените влияние на переобучение (снижение разницы потерь train/test).

Задание 3

Оценить влияние Batch Normalization.

Добавить слои нормализации после полносвязных слоев сначала к базовой модели, а потом к модели из второго задания. Сравнить результаты обучения.

Задание 4

Ранний стоп

1. Добавить к лучшей модели callback EarlyStopping с мониторингом валидационной ошибки.
2. Определить эффективное количество эпох.

Задание 5

Сформировать файл csv с отчетом по полученным результатам.

Метод	Точность на тесте (%)	Разница потерь (train–test)	Время обучения (мин)
Базовая модель			
L2-регуляризация			
Dropout			
Batch Normalization			

Задание 6

Реализовать подбор гиперпараметров для лучшей модели и оценить, как измениться качество обучения.

1. Определить следующее пространство поиска
   - скорость обучения ($\text{learning rate} = [0{,}001; 0{,}01; 0{,}1]$);
   - размер батча ($\text{batch size} = [32; 64; 128]$);
   - число нейронов в скрытых слоях ($[64; 128; 256]$);
   - коэффициент Dropout ($[0{,}2; 0{,}3; 0{,}5]$).
2. Применить два метода поиска
   a. **Grid Search**: полный перебор всех комбинаций (используйте GridSearchCV из scikit-learn).
   b. **Random Search**: случайный выбор комбинаций (количество итераций - 20).
3. В файл отчета дозаписать лучшие значение гиперпараметров, а также полученную при них точность, ошибку и время обучения.

Задание 7

Проверить стабильность лучшей модели с помощью кросс-валидации (k=3), найти среднюю точность модели

3 Материально-техническое обеспечение работы

Аудитория для проведения лабораторных занятий должна быть укомплектована специализированной мебелью и индивидуальными компьютерами следующей минимальной комплектации:

- Процессор: не менее двух исполнительных ядер, совместимый с системой команд x86 и x64, с поддержкой аппаратной виртуализации.
- Оперативная память: не менее 8 Гб.
- Монитор: не менее 24" (дюймов) по диагонали.
- Наличие локальной сети со скоростью обмена не менее 1 Гб/сек.
- Наличие доступа в сеть Интернет со скоростью не менее 1 Мбит/сек.
- Наличие клавиатуры и манипулятора «мышь».

На компьютерах должно быть установлено следующее программное обеспечение:

- Операционная система: Microsoft Windows 10 (или выше) и/или одна из операционных систем семейства Linux, допускающих установку среды исполнения для Python. Допускается конфигурация, когда одна из операционных систем установлена внутри виртуальной машины (гипервизора).
- Среды разработки и исполнения программ: Google Colab, PyCharm, VS Code.

4 Порядок выполнения и сдачи работы

Для выполнения лабораторной работы рекомендуется придерживаться следующего порядка выполнения:

1. Ознакомится с темой и целями лабораторной работы
2. Изучить теоретический материал
3. Подготовить рабочее окружение
4. Разработать программный код лабораторной работы в соответствие с заданием
5. Произвести проверку работоспособности, выполнить тестирование и отладку кода
6. Проанализировать полученные результаты и примененные в ходе решения подходы
7. Выполнить самооценку и рефлексию
8. Сдать лабораторную работу преподавателю и получить от него обратную связь

При сдаче студентом лабораторной работы основным отчетом выступает исходный код самостоятельно созданной в процессе выполнения работы программы.

Исходный код должен быть отформатирован согласно принятым для используемого языка программирования стандартам. Является желательным наличие в исходном коде комментариев, описывающих основные части программы и особенности их функционирования. В то же время студент должен быть готов объяснить работу программы в целом и каждую отдельную ее часть при полном отсутствии комментариев (например, они могут быть удалены преподавателем при сдаче работы).

Обязательным условием сдачи является умение студента самостоятельно восстановить любой участок исходного кода программы (но не более 20 строк подряд) после его удаления. Удаленный участок должен быть самостоятельно набран студентом заново в присутствие преподавателя. При восстановлении участка запрещается использовать операции Undo (Отменить) текстовых редакторов или переписывание кода участка из других источников.

Исходный код сдаваемой программы должен быть представлен в электронном виде.

Сдача исходного кода ранее сдававшихся программ или программ, код которых выложен в сети Интернет, не допускается.

5 Контрольные вопросы к лабораторной работе

1. Что такое переобучение и как его диагностировать?
2. В чём разница между L1 и L2 регуляризацией?
3. Почему Dropout снижает переобучение?
4. Что показывает кросс-валидация? В чём её преимущество перед одним разделением на train/test?
5. Чем отличается Grid Search от Random Search? Когда какой метод предпочтительнее?

6 Перечень использованных информационных ресурсов

1. Шолле Франсуа Глубокое обучение на Python. — СПб.: Питер, 2018. — 400 с.: ил. — (Серия «Библиотека программиста»).

2. Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. — СПб.: Питер, 2018. — 480 с.: ил. — (Серия «Библиотека программиста»).

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01.	В набор	В печать
Объем 0,5 усл.п.л., уч.-изд.л.	Офсет.	Формат 60x84/16.
Бумага тип №3.	Заказ №	Тираж 75. Цена

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:
344010, г. Ростов-на-Дону, пл. Гагарина, 1.